

Beyond Finite Closure

Certified Coverability and Nonlinear Recursive Probability

A design delta against Forge commit

58ea206bd7ad501b930add568151217bcc7f82a2

This report adds unbounded multiset-state safety, exact enclosures of irrational recursive probabilities, an inverse-free Newton certificate, and a critical-extinction certificate. It includes executable, standard-library-only searches and independently structured finite checkers. **Delivered evidence:** 2,220 coverability queries checked against complete finite forward exploration; 2,360 accepted receipts and 275 rejection controls in the main corpus; a separate 100-certificate compression experiment; deterministic reproduction and search-disabled replay. **Implementation boundary:** the algorithms and checkers were executed in Python. No Lean compilation, source-to-model adapter, integrated **forge** tactic, or head-to-head Lean benchmark was executed for this report.

Abstract

Forge’s merged design already contains structural proof planning, nonlinear arithmetic, finite-dimensional and ideal closures, finite-state quantitative reasoning, and nonprobabilistic pushdown summaries. Repeating those proposals would add little. This report crosses two of their remaining semantic boundaries: an unbounded natural-number state space and a nonlinear least fixed point whose value need not be rational.

The first extension computes a finite antichain describing the backward coverability set of a Petri net. A negative result is checked by finite predecessor-closure inequalities; a positive result is checked by an ordinary execution. The second computes rational lower and upper bounds for a probabilistic polynomial system. Lower bounds carry a rounded Newton ledger, not merely a fixed-point equation. A weighted maximum principle replaces full inverse matrices by two vectors per Newton step. A third certificate proves exact extinction probability one in a strongly connected critical or subcritical branching system, while rejecting the deterministic one-child exception. Finally, a factorization and rational isolating interval can identify an irrational least fixed point exactly.

The mathematics is based on established coverability and probabilistic-polynomial-system methods. The contribution is the concrete extension of Forge’s certificate language, the soundness decomposition, executable producers and checkers, and a narrowly specified Lean port. Search success, finite-checker acceptance, numerical diagnostics, source-level proofs, and compilation remain distinct throughout.

Contents

1	What this adds, and what it deliberately does not repeat	1
1.1	The actual baseline	1
1.2	Four new certificate forms	1
1.3	Established algorithms, concrete engineering contribution	2
1.4	What comes from Leant and Djex	2
2	Two infinite semantics, two different finite explanations	2
3	Unbounded multiset-state safety by backward antichains	3
3.1	The exact fragment	3
3.2	An exact predecessor formula	3
3.3	The search and its retained history	4
3.4	A safety certificate with no search trace	4
3.5	Why the unbounded search terminates	5
3.6	An infinite-state worked example	6
3.7	Extraction, abstraction, and future optimizations	6
4	Recursive probabilities as least fixed points	7
4.1	The polynomial-system fragment	7
4.2	A source semantics that really yields a product	7
4.3	Why the least solution is the relevant one	8
4.4	Upper bounds are easy; lower bounds need a history	8
4.5	A rational answer about an irrational quantity	9
5	Checked Newton ledgers and an inverse-free refinement	9
5.1	The inequality that licenses the step	9
5.2	The original exact-inverse certificate	9
5.3	Exact dyadic rounding and upper proposals	10
5.4	Removing inverse matrices from the certificate	10
5.5	Cost and measured certificate size	11
5.6	Optional local contraction and residual error	12
5.7	Exact algebraic identification without an algebraic-number oracle	12
6	Exact extinction at criticality	13
6.1	Why an interval engine is not the whole answer	13

6.2	The finite critical certificate	14
6.3	The deterministic one-child exception	15
6.4	A weight can succeed when row sums fail	15
6.5	Probability one is not finite expected work	15
6.6	Strongly connected components as a next extension	16
7	A concrete Lean implementation boundary	16
7.1	Do not start by porting the search	16
7.2	Finite data and semantic data should have different jobs	16
7.3	Suggested modules and their acceptance criteria	17
7.4	Libraries and exact roles	17
7.5	Source extraction algorithms	18
7.6	Tactic behavior and returned proof objects	19
7.7	Using the workers during term synthesis	19
8	Executed evidence and its limits	20
8.1	Environment and separation of runs	20
8.2	Main experiment matrix	20
8.3	Why the forward oracle is complete on its test fragment	21
8.4	What the probability cross-checks establish	21
8.5	Mutation and negative controls	21
8.6	Search-disabled replay	22
8.7	Inverse-free format measurements	22
8.8	What was not measured	22
9	Implementation order and falsifiable next milestones	22
9.1	First: one end-to-end unbounded safety theorem	22
9.2	Second: a weighted-step enclosure theorem	23
9.3	Third: exact critical and algebraic conclusions	23
9.4	Fourth: synthesis and larger semantic fragments	23
10	Assessment	24
A	Executable certificate protocol	24
A.1	Problem objects	24
A.2	Accepted receipt kinds	25
A.3	Input rejection and resource control	25
B	Checker-oriented proof dependency graph	25
C	Files, commands, and status ledger	26
C.1	Package contents	26
C.2	Replaying without search	26
C.3	Fresh production and comparison	26
C.4	Building the article	27
C.5	Evidence status	27
References		27

1 What this adds, and what it deliberately does not repeat

1.1 The actual baseline

The baseline is not the first proposal for a stronger version of **grind**. Forge at commit 58ea206bd7ad501b930add568151217bcc7f82a2 contains a 128-page merged article and twenty-seven archived proposals, arranged in three rounds. Its README distinguishes the unimplemented tactic from the Python algorithms and from the core-only Lean specimens subsequently elaborated by the merge. The latter distinction matters: it would be false to say that no Lean file in Forge has ever compiled. Conversely, their compilation does not constitute an implementation of the certificate checkers proposed here. These are the repository’s own statements, not experiments performed in this report [1].

The inspection covered the README, the merged conclusion, the quantitative section, the beginning of the closure section, the repository tree, and the existing Leant/Djex integration review. The neighboring repositories’ READMEs and current commit metadata were also inspected [2, 3, 4, 5, 6]. This is a targeted audit of the merged baseline, not a claim to have read every line of every archived submission. In particular, the absence claims below concern the capabilities described in that merged baseline. The exact files and revisions are retained in `SOURCE-AUDIT.md`.

The baseline already covers induction and generalization, witness synthesis, nonlinear arithmetic certificates, observable-space and ideal closures, finite algebraic covers, noncommutative identities, analytic remainder certificates, finite probabilistic systems, games, couplings, and nonprobabilistic pushdown summaries. It also already specifies scoped verification receipts, source ownership, worker separation, and disciplined negative evidence. None of these is presented as a new contribution here.

The strongest direct opening is in the baseline’s quantitative section. It excludes probabilistic pushdown and several other fragments when the result need not be an exact rational. That exclusion is a useful implementation boundary, but it is not a boundary of *exact certification*: rational enclosures can certify an irrational value, and a finite polynomial root description can sometimes identify it exactly [2, 9].

1.2 Four new certificate forms

Certificate	What it establishes	What makes it different from the baseline
Backward antichain	An upward-closed bad set cannot be reached in an unbounded multiset system.	The reachable state set need not be finite; termination of search comes from a well-quasi-order, not a fixed state bound or vector-space dimension.
Least-fixed-point enclosure	Rational ℓ, u with $\ell \leq \text{lfp}(P) \leq u$, optionally to a requested width.	Nonlinear recursive probability equations can have several fixed points and an irrational least solution.
Weighted Newton step	A new certified lower bound without a matrix inverse in the certificate.	A finite maximum-principle argument changes checker cost and certificate shape; it is implemented and measured here.
Critical extinction / isolated root	Either $\text{lfp}(P) = \mathbf{1}$, including a critical branching case, or an exact isolated algebraic root.	Strict contraction and rational-value solving do not express all these conclusions.

These are not four competing architectures. They are additional workers attached to the already proposed obligation controller. Their common pattern is unusually small: retain a finite object that certifies a fact about an infinite semantics. But their proof rules differ enough that they must not be conflated. An antichain is backward closed; a probability upper bound is a prefixed point; a probability lower bound needs a path from zero or another justified lower bound; and critical extinction needs a nondegeneracy argument.

1.3 Established algorithms, concrete engineering contribution

Backward coverability in well-structured systems and Newton methods for positive polynomial systems are established subjects [7, 10, 11]. This report does not claim to invent either. It derives the specific finite checks needed by Forge, proves their soundness in the precise fragments used by the prototypes, and exposes the places where a plausible generalization is false.

The distinction is useful for evaluating the work. A new name for an existing closure is not progress. A checker accepting an unbounded resource invariant, or a certified interval containing an irrational return probability, is a genuine change in the kinds of obligations the design can represent. Turning either into a Lean theorem still requires the explicitly listed source bridge and checker-soundness proofs.

1.4 What comes from Leant and Djex

Leant and Djex already preserve source-owned typed candidate information and separate successful reconstruction from unsupported negative claims. Forge’s own review already imports those lessons, so this report does not repackage them [4, 5, 6]. The additional connection proposed here is *behavioral*: a candidate using a small certified multiset or independent-branching API can be translated to a Petri or polynomial-system obligation, then analyzed by one of these workers.

A typed synthesis graph is useful provenance for that translation, not its correctness proof. A callback of a particular type need not have a particular transition effect. Two calls of the same function type need not use independent random choices. Each provider therefore needs a separate semantic law, and the adapter must preserve which provider and which law the candidate actually used. This creates a concrete route from typed synthesis to these new behavioral results without claiming that the current integer-indexing or two-universe synthesis gaps in Leant and Djex have been solved.

2 Two infinite semantics, two different finite explanations

The state space $\mathbb{N}^d$ is infinite even when d and the transition set are fixed. The lattice of real probability vectors $[0, 1]^n$ is infinite even when the polynomial system is finite. Neither difficulty is removed by testing more finite instances.

The coverability worker exploits an order on *states*. Once a marking can reach an upward-closed bad set, a larger marking can do so too. An entire infinite set can therefore have a finite basis. The probability worker exploits an order on *approximations*. Iterating a monotone polynomial map from zero approaches its least fixed point, and finite algebraic inequalities can bracket that point without enumerating recursive runs.

This contrast controls the implementation. Coverability produces a finite closure or a concrete trace. Probability approximation produces an interval, an exact special-case result, or a valid interval that is still too wide. A single Boolean “success” flag would erase useful distinctions. The prototype retains a requested-width result separately from validity; it does not invent a new system-wide outcome architecture in place of Forge’s existing one.

The mathematical carrier used below is explicit. All vector inequalities are componentwise. All certificate coefficients and all executed arithmetic are exact integers or rationals. Real numbers occur in the semantic theorems, not as floating-point inputs to the checkers. Numerical computations in the experiments are labeled diagnostic and do not authorize acceptance.

3 Unbounded multiset-state safety by backward antichains

Evidence status. Search, finite safety checker, execution checker, and differential experiments implemented and executed in Python. Lean source extraction and formal checker soundness are proposed, not executed.

3.1 The exact fragment

Definition 3.1 (Finite-place multiset transition system). Fix a dimension $d \geq 0$, an initial marking $m_0 \in \mathbb{N}^d$, a finite transition set T , and a finite set $B \subseteq \mathbb{N}^d$ of bad thresholds. Each $t \in T$ has input and output vectors $c_t, o_t \in \mathbb{N}^d$. The relation $m \xrightarrow{t} m'$ means

$$m \geq c_t, \quad m' = m - c_t + o_t.$$

For any $A \subseteq \mathbb{N}^d$, write

$$\uparrow A = \{m \in \mathbb{N}^d : \exists a \in A, a \leq m\}.$$

The question is whether a finite execution from m_0 reaches $\uparrow B$.

This is a Petri-net coverability question, not exact-state reachability. For example, a threshold $(0, 2, 0)$ means “at least two tokens in the second place,” not that the first and third places are empty. The dimensionality is fixed in each problem, but the number of tokens is not bounded. A model with N arbitrary participants can sometimes be represented through token counts; a model needing one distinct coordinate per participant does not automatically fit one fixed-dimensional instance.

Transitions with arbitrary zero tests, inhibitor arcs, upper guards, or resets do not satisfy this exact interface merely because their syntax mentions counters. The crucial law is monotonicity: if $m \xrightarrow{t} m'$ and $r \geq 0$, then $m + r \xrightarrow{t} m' + r$. The adapter must prove that law for the extracted source model or use an explicitly one-sided abstraction.

3.2 An exact predecessor formula

For a threshold b , a transition t can reach $\uparrow\{b\}$ exactly from markings above

$$\text{pb}_t(b) = c_t + \max(\mathbf{0}, b - o_t), \tag{1}$$

where the maximum is componentwise and the expression is interpreted in integers before returning a nonnegative vector. Equivalently, coordinate i is $\max(c_{t,i}, c_{t,i} + b_i - o_{t,i})$.

Lemma 3.2 (Minimal predecessor). *For every $m, b \in \mathbb{N}^d$,*

$$\text{pb}_t(b) \leq m \iff m \geq c_t \text{ and } m - c_t + o_t \geq b.$$

Proof. In each coordinate the right side is exactly the pair of inequalities $m_i \geq c_{t,i}$ and $m_i \geq c_{t,i} + b_i - o_{t,i}$. Their conjunction is the comparison against the maximum above. Coordinates are independent, so the equivalence holds for vectors. $\square$

This formula is the whole arithmetic content of the search. It also gives a useful independent test: compare it against operational enabling and firing, rather than implementing the same backward expression twice. The main corpus performs 3,350 such comparisons. The safety checker uses the maximum formulation while the producer uses the nonnegative-difference formulation; this is some implementation diversity, not formal independence.

3.3 The search and its retained history

Maintain an active antichain A : no two distinct active vectors are comparable. A new vector v is redundant when some active $a \leq v$. Otherwise admit v and remove every active a with $v \leq a$. A worklist expands each admitted active vector by all transitions.

```

A := empty active antichain
history := empty append-only array
for b in bad_thresholds:
    admit(b, terminal)
while an unprocessed active node remains:
    if a resource bound is exceeded: return unknown
    b := next active node
    for each transition t:
        v := pre[t] + max(0, b - post[t])
        if no a in A satisfies a <= v:
            admit(v, edge(t, historical_node_of(b)))
            if v <= initial: return its transition word
return the active basis with finite closure witnesses

```

Here `admit` removes dominated *active* entries but never deletes their history nodes. A new predecessor node points to the older node whose bad-reaching execution it extends. Following those pointers produces a finite word because every pointer goes to an older history node. The word is checked forward from the original initial marking, so no provenance claim is trusted by the positive checker.

Deleting history together with a dominated basis element would be an ordinary engineering bug: the basis could remain mathematically correct while a witness path became inaccessible or was accidentally reassigned with a different suffix. The active antichain and the append-only provenance DAG have different responsibilities and different lifetimes.

One other subtlety concerns worklist pruning. If b is replaced by a smaller active vector a , then $\text{pb}_t(a) \leq \text{pb}_t(b)$ for every transition. Expanding a therefore subsumes the predecessors that would have been produced from b . A stale worklist entry can be skipped, but only because the active set still represents a superset of its upward closure.

3.4 A safety certificate with no search trace

The negative answer is a basis $A = (a_0, \dots, a_{k-1})$ together with two small tables of indices. The checker verifies:

$$\forall b \in B, \quad a_{\tau(b)} \leq b, \quad \text{bad-set coverage,} \quad (2)$$

$$\forall i < k, \forall t \in T, \quad a_{\pi(i,t)} \leq \text{pb}_t(a_i), \quad \text{backward closure,} \quad (3)$$

$$\forall i < k, \quad a_i \not\leq m_0, \quad \text{initial exclusion.} \quad (4)$$

The indices must be in range. No rank, minimality, antichain property, search order, or claimed number of iterations is needed for soundness. A redundant basis is perfectly acceptable.

Theorem 3.3 (Finite safety certificate). *If (2)–(4) hold, no finite execution from m_0 reaches $\uparrow B$.*

Proof. Equation (2) implies $\uparrow B \subseteq \uparrow A$. Suppose $m \xrightarrow{t} m'$ and $m' \in \uparrow A$. Choose i with $a_i \leq m'$. Lemma 3.2 gives $\text{pb}_t(a_i) \leq m$, and (3) gives $a_{\pi(i,t)} \leq m$. Thus $m \in \uparrow A$. Consequently the complement of $\uparrow A$ is forward invariant. It contains m_0 by (4), and is disjoint from $\uparrow B$, proving the claim by induction on the execution length. $\square$

There is a reusable theorem hiding in this certificate. Once (2) and (3) are checked, the same A proves safety for *every* initial marking outside $\uparrow A$. The prototype binds each receipt to one initial marking, but a Lean port should expose the stronger invariant lemma and then specialize it. This is not a proposal to weaken subject identity: it changes the proposition proved by the generic checker theorem.

With supplied coverage indices, verification takes $O(d(k|T| + |B| + k))$ integer comparisons and elementary arithmetic operations. These are arithmetic-operation counts; large token thresholds still have a bit cost. Searching for the indices by linear scan belongs in the producer, not the checker.

3.5 Why the unbounded search terminates

Theorem 3.4 (Unbounded-budget coverability decision). *For the finite-place fragment above, backward antichain search with no resource cutoffs terminates. It returns either a valid safety basis or a finite execution covering a bad threshold.*

Proof. Each genuinely new admitted vector strictly enlarges the upward-closed set represented by the active basis. Removing dominated active vectors does not remove any marking from that set. Suppose infinitely many distinct admissions occurred, and list the admitted vectors in time order. If an earlier vector were below a later one, the later vector would already be covered by the current basis and could not be admitted. The sequence would therefore have no earlier element below a later element. Dickson’s lemma, the well-quasi-order property of componentwise comparison on $\mathbb{N}^d$, forbids such a sequence. Thus only finitely many vectors are admitted, and each contributes finitely many work items.

Each admitted threshold is a genuine backward predecessor of a bad threshold, with a finite transition suffix in its history. If an admitted threshold is below m_0 , Lemma 3.2 inductively makes that suffix executable from m_0 and bad-covering. If the search finishes without finding one, all predecessor thresholds are covered by the final basis, all bad thresholds remain covered, and m_0 is excluded. Theorem 3.3 applies. $\square$

The well-quasi-order argument is standard in backward coverability [7]. It is required to establish completeness of the *search*, not soundness of an individual accepted finite certificate. This separation is attractive for the first Lean milestone: the safety checker can be proved correct without formalizing Dickson’s lemma or the termination of an optimized search.

The theorem is not a practical small-time guarantee. Antichain sizes, threshold magnitudes, and witness lengths can be very large. The delivered implementation uses simple scans, explicit words, and fixed resource caps; hitting a cap produces **unknown**. A theorem saying an unbounded algorithm terminates does not entitle a bounded implementation to reinterpret exhaustion as safety.

3.6 An infinite-state worked example

Let the three places be available lock, active owner, and queued jobs. Use

$$m_0 = (1, 0, 0), \quad B = \{(0, 2, 0)\},$$

and the transitions

$$(1, 0, 0) \rightarrow (0, 1, 0), \quad (0, 1, 0) \rightarrow (1, 0, 0), \quad (0, 0, 0) \rightarrow (0, 0, 1).$$

The final transition can execute arbitrarily often, so the reachable set is infinite. A complete forward state enumeration does not finish. The backward worker returns

$$A = \{(0, 2, 0), (1, 1, 0), (2, 0, 0)\}.$$

For the acquire transition, the predecessors of these three thresholds are respectively $(1, 1, 0)$, $(2, 0, 0)$, $(3, 0, 0)$, all covered by A . For release they are $(0, 3, 0)$, $(0, 2, 0)$, $(1, 1, 0)$, again covered. Adding a queued job leaves the relevant predecessor thresholds unchanged. No element of A lies below $(1, 0, 0)$.

The resulting invariant says that the sum of available and held lock tokens is less than two. An existing linear-invariant worker could prove this example too; it is a transparent walkthrough, not evidence that every earlier Forge lane fails. The added capability is the complete backward coverability fragment and its finite certificate language, not a claim that this small invariant was previously inaccessible.

For a positive contrast, the one-place transition $0 \rightarrow 1$ from initial marking 0 covers the threshold 25. The producer returns a 25-step word; the checker executes those 25 steps. Setting the expansion budget to two returns **unknown**, not a nonreachability verdict. Both outcomes occur in the stored tests.

3.7 Extraction, abstraction, and future optimizations

A first source adapter should target a small explicit resource DSL rather than arbitrary Lean functions. A primitive consumes a statically known vector and produces a statically known vector. Sequential control can be encoded by extra control places. Alternatives become distinct transitions. Constant local tests whose enabling condition is exactly a lower threshold fit directly; zero tests do not.

For an abstraction $\alpha : S \rightarrow \mathbb{N}^d$, a safety transport proof needs three facts: the initial source state maps to the checked initial region, every concrete step is simulated by an abstract execution, and every concrete bad state maps into $\uparrow B$. This is sufficient for safety even when the net has extra abstract executions. It is not sufficient to turn an abstract bad-reaching word into a source counterexample. That direction needs a lifting proof or concrete replay against the source semantics.

The producer can later use dominance tries, coordinate indexing, partial-order reductions, or a specialized Petri solver. The checker remains unchanged if the result is the same basis and coverage table. This is preferable to trusting a complicated acceleration or pruning history. Research on Karp–Miller pruning contains concrete examples of plausible pruning procedures losing completeness [8]; the lesson here is not to reject acceleration, but to keep its output subject to a simple final invariant check.

Exact-state reachability, fairness, recurring visits, and deadlock-freedom under a scheduler require different predicates or semantics. They are not silently imported into a coverability result. A finite bad antichain is the input contract, and the report’s implementation stays inside it.

4 Recursive probabilities as least fixed points

Evidence status. Exact rational polynomial-system input, enclosure producers, finite checkers, and special-case certificates are executed. A probability semantics in Lean and the source compiler are not implemented by this package.

4.1 The polynomial-system fragment

Definition 4.1 (Probabilistic polynomial system). A probabilistic polynomial system (PPS) is a map $P : [0, 1]^n \rightarrow [0, 1]^n$, with $n \geq 1$, whose coordinates have finite presentations

$$P_i(x) = \sum_{\alpha \in E_i} p_{i,\alpha} x^\alpha, \quad x^\alpha = \prod_{j=1}^n x_j^{\alpha_j}, \quad p_{i,\alpha} \in \mathbb{Q}_{\geq 0}, \quad \sum_{\alpha} p_{i,\alpha} \leq 1. \quad (5)$$

Exponents are nonnegative integers. The semantic quantity of interest is the least fixed point $q = \text{lfp}(P)$ in the unit cube.

The coefficient restrictions are not incidental validation rules. They imply nonnegativity, monotonicity, and $P(\mathbf{1}) \leq \mathbf{1}$. The executable input parser merges duplicate monomials, removes zero terms, and rejects negative coefficients, negative exponents, inconsistent dimensions, and coefficient sums above one. Direct construction of a Python data object is revalidated at the public checker boundary; bypassing the JSON parser does not bypass these mathematical hypotheses.

4.2 A source semantics that really yields a product

Consider a finite collection of nonterminal or procedure types. An object of type i independently chooses one of finitely many rules. A rule of probability $p_{i,\alpha}$ creates α_j recursive children of type j for each j . A constant monomial describes immediate successful termination. Any missing probability mass is an explicit nonreturning outcome, not a normalization error to be repaired by dividing the coefficients.

For independent children, the probability that all children terminate is the product of their termination probabilities, giving (5). This covers multitype branching and appropriate stochastic grammars or single-exit state-free recursive models. General recursive Markov chains and stochastic grammars motivate these equations, including their irrational solutions [9]. This package does not implement a reduction from every probabilistic pushdown system.

A type signature does not establish the product law. Two child computations that consult the same random variable are correlated. Two procedures that communicate through mutable state may not have context-independent termination probabilities. A recursive call with an unbounded data parameter may require infinitely many unknowns rather than one variable per function name. Each of these must be rejected by the first adapter unless a separate sufficient semantic theorem is supplied.

Lean also cannot be given a possibly nonterminating recursive function and then be expected to treat its operational termination behavior as ordinary definitional reduction. The source-facing proposition should refer to an inductively defined finite derivation relation, an explicit probabilistic operational semantics, or a certified branching DSL. Its denotation is then connected to the polynomial system by a theorem. The algorithm does not make a nonterminating function into a total Lean definition.

4.3 Why the least solution is the relevant one

Set $q^{(0)} = \mathbf{0}$ and $q^{(k+1)} = P(q^{(k)})$. In the branching interpretation, $q_i^{(k)}$ is the probability mass of successful derivation trees whose height is at most k , with the precise convention that height zero has no successful root. Independence and finite alternatives prove the recurrence by induction.

The sequence is increasing and bounded above by $\mathbf{1}$. Its coordinatewise supremum exists, and polynomial continuity gives

$$P\left(\sup_k q^{(k)}\right) = \sup_k q^{(k+1)}.$$

Any fixed point $z \in [0, 1]^n$ bounds every $q^{(k)}$ by induction, so the supremum is the least fixed point. In a probability-space presentation, successful termination is the increasing union of these finite-height events, and continuity of probability from below identifies its probability with the same supremum.

The distinction between this least point and an arbitrary fixed point is fundamental. In

$$P(x) = \frac{1}{4} + \frac{3}{4}x^3, \tag{6}$$

1 is a fixed point. But the least fixed point is

$$q = \frac{\sqrt{21} - 3}{6} \approx 0.2637626158. \tag{7}$$

Returning 1 merely because it satisfies $P(x) = x$ would report an incorrect termination probability. Similarly, a lower candidate satisfying $\ell \leq P(\ell)$ is not automatically below the least fixed point: $\ell = 1$ is an immediate counterexample.

This is precisely why the lower certificates below retain a justified path from zero. Solving a polynomial equation, even exactly, does not select the semantic branch.

4.4 Upper bounds are easy; lower bounds need a history

Lemma 4.2 (Prefixed upper bound). *If $u \in [0, 1]^n$ satisfies $P(u) \leq u$, then $q \leq u$.*

Proof. The base approximation $q^{(0)} = \mathbf{0}$ is below u . If $q^{(k)} \leq u$, monotonicity gives $q^{(k+1)} = P(q^{(k)}) \leq P(u) \leq u$. Take the supremum. $\square$

Only an exact rational evaluation of $P(u)$ is needed for this upper certificate. An upper candidate obtained numerically, or from a linearized model, is harmless as a proposal provided that the original nonlinear inequality is checked afterwards.

A basic lower certificate is a finite ledger starting at zero, with steps

$$\ell \leq \ell' \leq P(\ell).$$

If $\ell \leq q$, then $\ell' \leq P(\ell) \leq P(q) = q$. Thus a finite Kleene ledger is sound. The monotonicity restriction $\ell \leq \ell'$ is convenient bookkeeping; the essential lower-bound inequality is the second one.

The problem is the number of steps, not the logic. At a critical branching point the error can decay slowly, while a Newton step can give much more information per certificate record. The next section proves exactly the stronger step we use, including the rounding direction and the matrix sign conditions.

4.5 A rational answer about an irrational quantity

An enclosure $[\ell, u]$ is an exact mathematical object even when it contains an irrational value. The user can ask for a threshold proof, an error bound, or a root identity. These are different queries.

For instance, $q < 1/3$ for (6) can be closed by a strict upper bound below $1/3$, without displaying any radical. A request for $|q - r| \leq 2^{-b}$ can be answered using a sufficiently narrow enclosure and a rational midpoint. A request for equality to (7) needs an additional algebraic identification. No approximate decimal is promoted to exact equality.

The prototype records `requested_bits` and recomputes the Boolean `target_met` from

$$\max_i (u_i - \ell_i) \leq 2^{-b}.$$

A valid interval $[0, 1]$ with an unmet precision request is still valid information; it is not a completed approximation request. An intentionally false `target_met` flag is rejected by the checker. This gives a simple, mechanical distinction between valid partial progress and a finished quantitative task.

5 Checked Newton ledgers and an inverse-free refinement

5.1 The inequality that licenses the step

Let $J(\ell)$ be the Jacobian of the polynomial map at a rational vector ℓ . Its entries are nonnegative when $\ell \geq 0$. The key inequality is directional, not an assertion that a multivariate polynomial is convex.

Lemma 5.1 (Nonnegative directional remainder). *For $0 \leq x \leq y$,*

$$P(y) \geq P(x) + J(x)(y - x). \quad (8)$$

Proof. Write $y = x + h$ with $h \geq 0$. Expand each monomial in the coordinates of $x + h$. Its terms of total degree zero in h give the original monomial at x ; those of total degree one give its derivative applied to h . Every remaining term has a nonnegative coefficient and nonnegative factors. Discarding those terms proves the inequality monomial by monomial. Multiply by the nonnegative input coefficients and sum. $\square$

The polynomial xy is not globally convex on the positive quadrant: its Hessian has a negative eigenvalue. It nevertheless satisfies (8) on comparable nonnegative vectors. The proof above is what a Lean port should formalize. Invoking a nonexistent global convexity fact would be both unnecessarily difficult and wrong.

5.2 The original exact-inverse certificate

Assume $\ell \leq q$. Put

$$A = I - J(\ell), \quad r = P(\ell) - \ell.$$

A producer supplies a rational matrix R , checks $R \geq 0$ and $RA = I$, and forms $\delta = Rr$. It proposes a rational next lower bound satisfying

$$\delta \geq 0, \quad \ell \leq \ell' \leq \ell + \delta. \quad (9)$$

The receipt contains R and ℓ' ; the checker recomputes $J(\ell)$, r , and δ .

Theorem 5.2 (Rounded Newton lower step). *If $R \geq 0$, $R(I - J(\ell)) = I$, and (9) hold, then $\ell' \leq q$.*

Proof. Apply Lemma 5.1 to $\ell \leq q$ and use $P(q) = q$:

$$q \geq P(\ell) + J(\ell)(q - \ell), \quad (I - J(\ell))(q - \ell) \geq P(\ell) - \ell = r.$$

Left multiplication by the entrywise nonnegative matrix R preserves componentwise inequalities. The checked identity then gives $q - \ell \geq Rr = \delta$. Therefore $\ell' \leq \ell + \delta \leq q$. $\square$

This proof does not trust an inversion routine, floating-point conditioning test, or iteration count. The checker multiplies matrices and compares rationals. It does not need to prove that the producer followed any particular Newton implementation.

The nonnegative-inverse condition is essential. An arbitrary algebraic inverse does not preserve the inequality used in the proof. The nonnegative-direction condition also has an operational role: it lets the ledger remain monotone under downward rounding. The search rejects a Newton proposal that fails these checks and may use a justified Kleene step instead.

5.3 Exact dyadic rounding and upper proposals

The implemented search uses exact Gauss–Jordan elimination to propose R . To keep the lower coordinates small, it rounds $\ell + \delta$ *down* to multiples of 2^{-h} . Because the old ℓ already lies on that grid and $\delta \geq 0$, the rounded point stays above ℓ . Rounding upwards would require a new proof and is not permitted by this step rule.

After a lower update the producer tries upper candidates of the form

$$u = \ell + 2^{-k}w, \quad w = (I - J(\ell))^{-1}\mathbf{1},$$

when that direction is positive, and otherwise tries $w = \mathbf{1}$. It keeps only candidates inside the unit cube that satisfy the original test $P(u) \leq u$. The set of tested k is finite. Failure to find a close prefixed point says that this search has not found one, not that none exists.

Previously accepted upper bounds can be combined by coordinatewise minimum. If u and v are prefixed, monotonicity gives $P(u \wedge v) \leq P(u) \wedge P(v) \leq u \wedge v$. Thus this refinement is sound even if the two upper vectors were proposed by different heuristics.

The dyadic lower grid does not imply all certificate numbers are small dyadics. Inverse directions and prefixed upper candidates can acquire large rational denominators. The stored cubic example illustrates this: its lower endpoint is a modest dyadic rational, while the upper endpoint and contraction coefficient have much larger numerators and denominators. Exact arithmetic removes rounding ambiguity; it does not remove coefficient growth.

5.4 Removing inverse matrices from the certificate

The full inverse is often unnecessary proof data. A weighted maximum principle gives a smaller certificate and a simpler local checker.

Lemma 5.3 (Finite weighted maximum principle). *Let $J \in \mathbb{R}_{\geq 0}^{n \times n}$ and let $w \in \mathbb{R}_{> 0}^n$ satisfy $Jw < w$ in every coordinate. If $(I - J)z \geq 0$, then $z \geq 0$.*

Proof. Suppose some coordinate of z is negative. Choose i minimizing z_i/w_i , and call that minimum $c < 0$. Then $z_j \geq cw_j$ for every j . Nonnegativity of J gives $(Jz)_i \geq c(Jw)_i$, so

$$((I - J)z)_i = cw_i - (Jz)_i \leq c(w_i - (Jw)_i) < 0.$$

This contradicts $(I - J)z \geq 0$. $\square$

The proof only uses a finite minimum, sums, products, and inequalities. It does not require spectral-radius theory, matrix inversion, normed spaces, or an analytic convergence theorem.

Theorem 5.4 (Inverse-free Newton certificate). *Suppose $0 \leq \ell \leq q$. A certificate supplies rational vectors $w > 0$ and $d \geq 0$ satisfying*

$$J(\ell)w < w, \quad (I - J(\ell))d \leq P(\ell) - \ell. \quad (10)$$

Then every ℓ' with $\ell \leq \ell' \leq \ell + d$ satisfies $\ell' \leq q$.

Proof. The remainder inequality gives

$$(I - J(\ell))(q - \ell) \geq P(\ell) - \ell \geq (I - J(\ell))d.$$

Apply Lemma 5.3 to $z = q - \ell - d$ to obtain $q - \ell - d \geq 0$. The desired upper bound on ℓ' follows. $\square$

For an ordinary accepted exact Newton step, take $d = Rr$ and $w = R\mathbf{1}$. Since the matrix is square, the verified left inverse is its two-sided inverse; thus $(I - J)w = \mathbf{1}$, giving strictness, and $(I - J)d = r$. Each row of the invertible nonnegative matrix R is nonzero, so $w > 0$. This proves that the new format can represent the existing accepted inverse steps. The converter does not rely on that argument for acceptance: it emits the vectors and the finite checker rechecks (10) directly.

Do not round the residual equation in the wrong direction. If d solves $(I - J)d = r$, rounding d down componentwise does *not* necessarily preserve $(I - J)d \leq r$, because $I - J$ has negative off-diagonal entries. The delivered converter keeps the exact direction d and rounds only the proposed endpoint ℓ' . The checker verifies $\ell' \leq \ell + d$ separately. This small distinction is important for a sparse or numerical future producer.

Strictness is not optional. The weak condition $Jw \leq w$ is insufficient for this lower-step rule. For $P(x) = x$, take $\ell = 0$, $w = 1$, and $d = 1$. The residual inequality is $0 \leq 0$, and a weak weight test would accept $\ell' = 1$ even though the least fixed point is zero. A stored rejection control exercises exactly this forgery. Critical equality belongs to the different theorem in Section 6, where nondegeneracy supplies the missing argument.

5.5 Cost and measured certificate size

For a dense $n \times n$ Jacobian, checking $R(I - J) = I$ takes $O(n^3)$ rational arithmetic operations and stores n^2 inverse entries. The weighted step uses two matrix-vector products, taking $O(n^2)$ operations and storing two length- n vectors, besides the next endpoint. Polynomial evaluation and Jacobian construction are additional costs common to both forms.

A sparse checker should never materialize the full Jacobian merely to compute Jw and Jd . For each input monomial, differentiate each variable with a positive exponent and accumulate its weighted contribution. Prefix/suffix products of factors avoid repeated full products and avoid division by a possibly zero coordinate. This is a concrete production optimization; the prototype uses straightforward exact loops instead.

The executed converter translated 100 accepted enclosure receipts containing 471 Newton steps. All converted receipts passed, as did 201 rejection controls. With compact JSON serialization, total receipt bytes fell from 297,913 to 228,676, about 23.2%. This is not a universal size reduction: dimension-one receipts grew from 41,410 to 55,387 bytes, whereas dimension-four receipts shrank

from 162,855 to 97,871 bytes. A production exporter should compare actual serialized sizes and choose the cheaper admissible form. These measurements concern the same source problems in two proof formats; they are not 100 additional solved problems.

5.6 Optional local contraction and residual error

A checked enclosure can optionally carry $w > 0$ and a rational $0 \leq \rho < 1$ with

$$J(u)w \leq \rho w. \quad (11)$$

Since polynomial derivatives have nonnegative coefficients, $J(x) \leq J(u)$ throughout $[0, u]$. In the weighted sup norm

$$\|z\|_w = \max_i |z_i|/w_i,$$

this bounds the Lipschitz constant of P on $[0, u]$ by ρ . Hence there is at most one fixed point in that box, and for $x \in [0, u]$,

$$\|q - x\|_w \leq \frac{\|P(x) - x\|_w}{1 - \rho}.$$

The last inequality follows by adding and subtracting $P(x)$ in $q - x = P(q) - x$, using the Lipschitz bound, and rearranging.

This is an optional strengthening, not the foundation of every enclosure. In (6), the derivative at 1 is $9/4$, so the global interval $[0, 1]$ has no scalar strict contraction witness. But on $[0, 1/3]$ the derivative is at most $1/4$. The local upper certificate opens a contractive region that a global contraction test would miss. The delivered cubic ablation obtains a checked local contraction after three Newton lower steps.

Literature on rounded and decomposed Newton methods establishes much stronger complexity and approximation results under carefully specified hypotheses [10, 11, 12]. The prototype here is a small exact certificate producer, not an implementation of all those algorithms and not a proof of their polynomial-time bounds. Its fixed precision, finite upper-candidate family, dense elimination, and step budget can all cause it to return an enclosure that misses the requested width.

5.7 Exact algebraic identification without an algebraic-number oracle

For a scalar system, suppose an accepted enclosure puts q in a rational interval $[\ell, u]$. A producer supplies rational polynomials g, h and the checker verifies

$$P(X) - X = g(X)h(X). \quad (12)$$

It then evaluates h by rational interval arithmetic on $[\ell, u]$ and checks that zero is excluded. It also interval-evaluates g' and checks that its sign is strictly positive everywhere or strictly negative everywhere on the interval.

Theorem 5.5 (Isolated least-root certificate). *Under these checks, q is the unique zero of g in $[\ell, u]$.*

Proof. The enclosure and least-fixed-point semantics give $q \in [\ell, u]$ and $P(q) - q = 0$. The factor identity and $h(q) \neq 0$ imply $g(q) = 0$. The derivative sign makes g strictly monotone on the interval, so it has at most one zero there. Existence came from the least fixed point, not from a numerical root finder. $\square$

The derivative argument is a small scalar analysis theorem to prove once in Lean. The interval calculations themselves are rational Horner evaluations with exact endpoint bounds. The factor need not be irreducible or a minimal polynomial. This format identifies a real number; it does not claim a canonical algebraic-number representation.

For (6),

$$P(X) - X = \frac{1}{4}(X - 1)(3X^2 + 3X - 1).$$

Take $[\ell, u] = [1/4, 1/3]$. The lower bound $1/4$ is one Kleene step, and the upper bound satisfies

$$P(1/3) = \frac{1}{4} + \frac{1}{36} = \frac{5}{18} < \frac{1}{3}.$$

The cofactor $X - 1$ stays negative, and $(3X^2 + 3X - 1)' = 6X + 3$ stays positive. Thus the least probability is the root displayed in (7). This entire identification is certified with rational arithmetic plus generic soundness lemmas; no decimal root is trusted.

The prototype's decoder is intentionally narrower than a complete real-root isolation package. Repeated roots, a cofactor interval crossing zero because of dependency overestimation, or an interval derivative containing zero can cause rejection even when the desired identity is true. Reusing Forge's existing scalar sign or Sturm machinery is the natural next adapter, rather than proposing that machinery again as a new contribution.

6 Exact extinction at criticality

Evidence status. A finite exact checker and a rational-weight search are implemented. The theorem below is proved on paper; no Lean proof of it was compiled.

6.1 Why an interval engine is not the whole answer

Consider

$$P(x) = \frac{1}{2} + \frac{1}{2}x^2. \tag{13}$$

Its least fixed point is 1, and $P'(1) = 1$. There is no strict contraction certificate on an interval containing the fixed point. The polynomial $P(X) - X$ has a repeated root at 1, so the derivative-sign root decoder also does not apply directly.

Newton lower bounds still work below the root. If $\ell = 1 - e$ with $e > 0$, then $P(\ell) - \ell = e^2/2$ and $1 - P'(\ell) = e$, so the Newton increment is $e/2$. Each exact Newton step halves the error. Twenty steps from zero give the exact lower bound $1 - 2^{-20}$.

The ordinary iteration satisfies

$$e_{k+1} = e_k - \frac{1}{2}e_k^2,$$

which is much slower near zero. In the executed dyadically rounded ablation, 256 Kleene steps left an error of

$$\frac{8350524623}{1099511627776} \approx 0.007595,$$

whereas twenty Newton steps met the 2^{-20} width request. A separate exact-extinction certificate proves $q = 1$ without approximation steps. This is a comparison between three restricted mechanisms on one stated example, not a benchmark against Lean tactics.

6.2 The finite critical certificate

Assume now that every row is normalized: $P(\mathbf{1}) = \mathbf{1}$. Let

$$M = J(\mathbf{1})$$

be the nonnegative mean-offspring matrix. Form the directed dependency graph with an edge $i \rightarrow j$ whenever $M_{ij} > 0$.

The certificate supplies a positive rational vector w . The checker verifies

$$Mw \leq w, \tag{14}$$

strong connectivity of the graph, and the following nondegeneracy condition:

$$\begin{aligned} &\text{some row has } (Mw)_i < w_i, \quad \text{or} \\ &\text{some monomial of degree at least two has positive coefficient.} \end{aligned} \tag{15}$$

Strong connectivity is checked by reachability from vertex zero in the graph and its reversal. This is a finite graph calculation. The checker does not compute eigenvalues or trust a spectral-radius estimate.

Theorem 6.1 (Critical or subcritical extinction). *If $P(\mathbf{1}) = \mathbf{1}$, the dependency graph is strongly connected, and (14)–(15) hold with $w > 0$, then $\text{lfp}(P) = \mathbf{1}$.*

Proof. Let $q = \text{lfp}(P)$ and put $d = \mathbf{1} - q \geq 0$. For numbers $z_1, \dots, z_k \in [0, 1]$, the elementary product inequality

$$1 - \prod_{r=1}^k (1 - z_r) \leq \sum_{r=1}^k z_r \tag{16}$$

follows by induction. If at least two of the z_r are strictly positive, the inequality is strict: the two-factor difference is their positive product, and adjoining further factors cannot erase that positive deficit.

Using the exponent vector as a list of repeated factors, row normalization and $P(q) = q$ yield

$$d_i = \sum_{\alpha} p_{i,\alpha} (1 - (\mathbf{1} - d)^\alpha) \leq \sum_j M_{ij} d_j.$$

Thus $d \leq Md$.

Suppose $d \neq 0$, and let $c = \max_i d_i/w_i > 0$. Choose a maximizing index i . Then

$$cw_i = d_i \leq (Md)_i \leq c(Mw)_i \leq cw_i.$$

All inequalities are equalities. Since every summand is nonnegative, equality in the middle inequality forces $d_j = cw_j$ for each outgoing neighbor j of i . Such a neighbor is itself maximizing, so the argument propagates along paths. Strong connectivity gives $d = cw$ in every coordinate; in particular, all d_j are positive.

A strict row in (14) now contradicts $d \leq Md$. Otherwise, a positive-coefficient monomial of degree at least two makes (16) strict in its row, because it contains at least two strictly positive deficits, counting repeated occurrences. This gives $d_i < (Md)_i \leq cw_i = d_i$, again a contradiction. Hence $d = 0$ and $q = \mathbf{1}$. $\square$

The theorem is a finite-weight formulation of the critical/subcritical branching argument, consistent with the standard least-fixed-point treatment of multitype branching [11]. The proof is included because it defines exactly the certificate assumptions, including the exception that a spectral slogan can easily omit.

6.3 The deterministic one-child exception

For $P(x) = x$, the least fixed point is zero: iteration from zero never leaves zero. Yet $P(1) = 1$, $M = (1)$, and $Mw = w$ for every positive w . The dependency graph is strongly connected. Without (15), a purported critical-extinction checker would therefore prove something false.

The same problem occurs for a deterministic cycle of types:

$$P_1(x) = x_2, \quad P_2(x) = x_3, \quad \dots, \quad P_n(x) = x_1.$$

There is exactly one child forever and no finite successful derivation. The main tests include four such systems and reject forged probability-one receipts for all of them. The inverse-free Newton experiment additionally rejects a forged lower step on the one-variable identity map whose weight inequality is nonstrict.

More generally, normalized linear systems can randomize which single child is created and still create exactly one forever. If there are no terminating constants and no branching monomials, the critical equality case must not be accepted just because the type graph is irreducible. Nondegeneracy is a mathematical hypothesis, not a heuristic filter.

6.4 A weight can succeed when row sums fail

Take

$$P_1(x, y) = \frac{1}{10} + \frac{9}{10}y^2, \quad P_2(x, y) = \frac{4}{5} + \frac{1}{5}x.$$

Then

$$M = \begin{pmatrix} 0 & 9/5 \\ 1/5 & 0 \end{pmatrix}.$$

The first row sum exceeds one, so the all-ones weight fails. But $w = (2, 1)^\top$ gives

$$Mw = (9/5, 2/5)^\top < w.$$

The graph is strongly connected and the rows are normalized. The theorem proves both extinction probabilities equal one.

The producer tries a small exact family of weights: $\mathbf{1}$; $(I - M)^{-1}\mathbf{1}$ when the inverse exists; and signed vectors from an exact nullspace basis of $I - M$. It sends every candidate through the checker. This is not a general linear-programming feasibility implementation. It can miss a usable positive vector in a broader setting, and failure produces **unknown**. For a production solver, rational linear programming with $w_i \geq 1$ and $Mw \leq w$ is a concrete additional proposer; the final certificate remains just w .

6.5 Probability one is not finite expected work

For (13), an individual has zero or two children with equal probabilities. Its expected number of children is one. Starting with one root, the expected population in every generation is therefore one, even though extinction occurs with probability one. The expected total number of individuals is the sum of those expectations and diverges.

Accordingly, an **extinction_one** receipt has only a probability-one conclusion. It does not establish finite expected runtime, a finite worst-case bound, or termination on every possible sample sequence. These require separate quantitative or pathwise arguments. The current receipt type is deliberately not named **terminates**.

6.6 Strongly connected components as a next extension

The implemented checker treats a whole strongly connected system. A production worker should decompose a general dependency graph into strongly connected components and process dependencies first. A downstream component already proved equal to zero or one may be substituted by that *exact* value, with a recorded theorem justifying the substitution.

An interval enclosure of a downstream component is different. Substituting its upper endpoint changes the polynomial map and generally gives an upper abstraction, not an equal system. Lower and upper substitution maps must be related to the original least fixed point by a separate monotonicity theorem. One cannot call this ordinary SCC elimination and then use the resulting point values as equalities.

This is a precise next algorithmic task: implement a component DAG, retain exact solved coordinates when available, build lower and upper polynomial maps for uncertain dependencies, and attach monotone parameter-transport lemmas. It is designed here but is not part of the executed corpus.

7 A concrete Lean implementation boundary

7.1 Do not start by porting the search

The first Lean implementation should port the finite checker and its soundness theorem, leaving search in Python or an external exact backend. None of the proposed final theorems should depend on the searcher’s termination, its antichain minimality, its elimination algorithm, or its numerical heuristics.

This is a specific implementation choice rather than another restatement of Forge’s trust architecture. For the antichain lane, the proof kernel needs finite coordinate comparisons plus a generic induction on executions. For weighted Newton, it needs finite sums, the polynomial remainder lemma, and the weighted maximum principle. The matrix-inverse algorithm can disappear entirely from the Lean code for that format.

No Lean executable or Lake installation was available in this execution environment, and attempted network acquisition of a toolchain failed. Therefore this package contains no uncompiled theorem file presented as an implementation. It instead includes `lean/IMPLEMENTATION.md` with the exact proof obligations and module plan. The status is `NOT_RUN`; this says nothing negative about the core specimens already compiled in the baseline repository.

7.2 Finite data and semantic data should have different jobs

For Petri certificates, use a finite dimension and vectors of natural numbers. An array representation is convenient for evaluation, while the semantic vector type can be a function `Find->Nat`. Prove once that the checked array projection has the stated dimension and that each coordinate comparison agrees with the semantic relation.

For probabilities, keep the checker representation as sparse rational monomials. A term contains a nonnegative coefficient and an exponent vector. The semantic interpretation maps it to a real polynomial on the unit cube. There is no need to ask the kernel to execute a general symbolic differentiation package: the Jacobian entries are explicit finite sums over monomials, and their agreement with the directional linear term can be proved algebraically.

The central semantic interface can be expressed mathematically as

$$\text{ValidPPS}(P), \quad q = \text{lfp}(P), \quad \text{checkEnclosure}(P, c) = \text{true} \quad \implies \quad \forall i, \ell_i \leq q_i \leq u_i.$$

The checker should return a decoded structure, or its caller should retain the exact decoded input, so that the P , ℓ , and u in the theorem are those actually checked. Forge already has subject-identity requirements; this port should reuse them rather than introduce a second authority-bearing pretty-print layer.

7.3 Suggested modules and their acceptance criteria

All module names in this table are relative to the proposed `Forge/Unbounded/` directory.

Proposed module	Required theorem or executable boundary
<code>PetriModel</code>	Operational step relation, upward closure, and Lemma 3.2.
<code>PetriCheck</code>	Natural subtraction must remain guarded by enabling. Total finite checker for (2)–(4); theorem 3.3; separate forward word checker.
<code>PPSModel</code>	Valid sparse polynomial data, nonnegative evaluation, monotonicity, mapping of the unit cube into itself.
<code>PPSLfp</code>	Least fixed point on the cube and agreement with finite-height approximants. Keep the latter separate from purely order-theoretic fixed-point facts.
<code>WeightedStep</code>	Directional remainder lemma, weighted maximum principle, and Theorem 5.4.
<code>EnclosureCheck</code>	Ledger induction, prefixed upper check, exact requested-width comparison.
<code>ExtinctionCheck</code>	Normalization, graph connectivity, positive weights, nondegeneracy, and Theorem 6.1.
<code>RootDecode</code>	Polynomial factor identity, interval Horner correctness, strict derivative sign, and Theorem 5.5.
<code>SourceBridge</code>	Source-specific simulations or branching probability equations. This is a required proof layer, not just serialization.

A first accepted milestone is one source-level resource theorem whose proof term applies `PetriCheck` soundness to a checked certificate and a proved source simulation. A second is a theorem about a declared PPS least fixed point. A third connects that PPS theorem to a source stochastic program. These are intentionally separate milestones; compiling the finite arithmetic layer alone does not close the source theorem.

7.4 Libraries and exact roles

The recommended dependencies have different authority and execution roles. The Python prototype requires none beyond the standard library. The external libraries below are implementation candidates, not dependencies that were installed or benchmarked here.

Mathlib fixed points. The current generated documentation for `Mathlib.Order.FixedPoints` exposes `OrderHom.lfp`, `OrderHom.lfp_le`, `OrderHom.map_lfp`, and `fixedPoints.lfp_eq_sSup_iterate` [13]. Use the order-homomorphism least fixed point on a complete-lattice presentation of the unit cube, not on all of $\mathbb{R}^n$, which is not a complete lattice. The iteration theorem requires its continuity hypothesis; monotonicity alone does not establish that countably many iterations reach an arbitrary lattice least fixed point.

Mathlib well-quasi-orders. The documentation for `Mathlib.Order.WellFoundedSet` provides `Set.PartiallyWellOrderedOn`, finite-product support, and finite-antichain results; it imports `Mathlib.Order.WellQuasiOrder` [14]. These are appropriate for a later formal completeness theorem for the antichain search. They are not prerequisites for checking an individual invariant basis.

Rational matrix and polynomial search. FLINT’s `fmpq_mat` is an exact rational-matrix substrate and internally supports integer-based work after clearing denominators [15]. Its `fmpq_poly` module represents sparse rational multivariate polynomials [16]. A production producer can use these for Jacobian construction and exact solves, export rational vectors or matrices, and leave checking to Lean. The present prototype instead implements dense rational elimination directly, making its behavior and limitations easy to inspect.

Scalar algebraic proposals. After clearing denominators in $P(X) - X$, FLINT’s documented `fmpz_poly_factor` function supplies integer polynomial factors. Its `qqbar` subsystem represents algebraic numbers with minimal polynomials and isolating intervals, and offers `qqbar_roots_fm` `pz_poly` [17, 18]. These are useful producer-side tools for choosing a factor and interval. Their output is not a Lean proof. The small decoder here checks a rational factor identity and interval signs and does not import an entire algebraic-number implementation into the trusted path.

Petri model checking. SMPT is an existing SMT-based Petri-net model checker and is a concrete candidate for independent benchmarks or source-model interchange [19]. Its availability is not evidence that it already exports this report’s certificate schema. An adapter would have to extract or synthesize the final basis, or translate an invariant into another Forge certificate form. Solver verdict strings are not interchangeable with the supplied finite checks.

Version discipline. Forge’s reviewed README pins Lean 4.34.0 and a specific Mathlib revision. The API names above were checked against current generated documentation, not compiled against that pin. Before implementation, inspect the same modules at the chosen Mathlib revision and run the actual build. The report does not silently equate a current documentation page with a compiler-verified dependency lock.

7.5 Source extraction algorithms

Multiset programs. Start from normalized operations `consumev`, `producev`, finite choice, and finite control-state transitions. Symbolically accumulate fixed resource effects between control points. A basic block becomes one Petri transition when its exact enabling requirement is a componentwise lower bound. A branch whose condition is not preserved by adding resources must either remain outside the fragment or be weakened under an explicit simulation theorem.

Each emitted transition carries its source block identifier and a proof obligation: executing that block from any source state satisfying the block precondition produces the abstract marking described by the transition. For source paths represented by several net steps, use a reflexive-transitive simulation rather than pretending every abstract step has the same source duration.

Independent recursive branching. Use a finite typed grammar of rules. A normalized rule has an exact rational probability and a finite list of child nonterminal identifiers. Count identifiers to obtain its exponent vector and merge rules yielding the same monomial. The source law for a rule states that all children succeed independently and that the selected alternative has the

given probability. A rule’s missing probability mass is represented as explicit failure/nonreturn, not silently discarded.

The resulting polynomial compiler is straightforward once these laws exist. Its hard theorem is agreement between source finite-height derivations and iterates of the compiled map. Prove the base case, prove the one-generation product/sum step, and only then pass to the increasing limit. This isolates the independence assumption where it belongs.

7.6 Tactic behavior and returned proof objects

The proposed user-facing syntax is schematic, not installed syntax:

```
by forge (lane := coverability)      -- resource safety goal
by forge (lane := probability) (bits := 40)
by forge (lane := extinction)        -- probability equals one
```

The implementation should recognize the corresponding semantic heads or an explicitly registered bridge theorem, construct the finite problem, request a certificate, check it, and apply the lane’s soundness theorem. Ordinary `grind` can discharge local side conditions or use the resulting inequalities; it is not asked to reproduce the unbounded search.

For a threshold query $q_i < r$, the returned proof should use a checked $u_i < r$. For a nonstrict query $q_i \leq r$, an equality upper endpoint may suffice. If the interval straddles r , increasing precision is a search choice, not a theorem. If exact equality is requested, use the extinction or algebraic route rather than fabricating equality from a narrow interval.

The final stored proof should contain the finite certificate, a checked decoding result, the generic soundness theorem, and the source bridge. Recompile then avoids the synthesizer. The existing Forge architecture already calls for this separation; the contribution here is specifying the actual mathematical objects that can occupy those slots.

7.7 Using the workers during term synthesis

For a Leant/Djex candidate assembled from certified providers, the adapter can derive a behavioral model from the candidate’s retained source-owned graph. A resource candidate receives a safety query; a recursive probabilistic candidate receives a least-fixed-point threshold query. A concrete source replay can refute a bad resource candidate, while a checked enclosure can establish a quantitative contract for another.

Do not prune a candidate merely because an *overapproximating* Petri abstraction is coverable. Do not reject a probabilistic candidate merely because an interval is too wide. Do not assume a graph’s typed equivalence entails behavioral equivalence for stateful or probabilistic providers. These are not new global trust rules; they are the concrete approximation polarities of the proposed adapters.

An attractive first synthesis experiment is a small inventory of resource protocols with known semantic providers: synthesize the control composition, compile its net, and compare the accepted source theorem or concrete counterexample against the candidate’s original contract. This is future integration work. None of the present Python cases is reported as a Leant or Djex behavioral acceptance.

8 Executed evidence and its limits

8.1 Environment and separation of runs

The main suite ran under Python 3.13.5 on Linux, using only the standard library. The fixed seed was 6071504. The recorded environment is in `results/environment.json`. Search and checking use exact `fractions.Fraction` arithmetic; no NumPy, SciPy, SymPy, SMT solver, or algebra package is needed to run or replay the package.

The main corpus, the mechanism ablations, and the inverse-free conversion experiment have separate directories and separate counts. The second main run is a reproduction of the first, not another independent collection of mathematical cases. Its accepted and rejected JSONL files were byte-identical to the first run. The report never adds these figures to Forge’s twenty-seven prior runs.

8.2 Main experiment matrix

Group	Count	Outcome and comparison basis
Predecessor equivalence	3,350	Formula compared with operational enabling and firing; all comparisons agree.
Exhaustive conservative Petri cases	2,100	Complete forward BFS: 1,536 safe and 564 coverable; all match the backward worker.
Random conservative Petri cases	120	Complete forward BFS: 79 safe and 41 coverable; all match.
Unbounded and edge Petri cases	6	Five accepted safety/trace receipts and one budget-unknown control; includes the infinite-state mutex.
Scalar quadratic PPS	45	Exact oracle $q = \min(1, a/b)$; all 20-bit width requests met; at most 20 Newton steps.
Coupled contractive PPS	30	All 22-bit widths met; at most four Newton steps; 80-digit iteration is a diagnostic cross-check.
Coupled supercritical PPS	24	All 22-bit widths met; symmetric scalar algebraic values provide numerical diagnostics.
Spectral extinction	26	Exact probability-one certificates, including critical cycles and a nonuniform positive weight.
Algebraic decoder	8	Accepted isolated scalar least-root descriptions.
Quantitative boundary cases	6	Four singular one-child exceptions, a critical noncontractive system, and a valid but insufficiently narrow enclosure.
Rejection controls	275	All deliberately invalid or non-authoritative receipts rejected.
Malformed input controls	5	All rejected by input validation.

The main corpus contains 2,360 accepted stored receipts and 275 rejected stored receipts. These are *receipt counts*, not a count of distinct source-level theorems. Some systems occur in more than one certificate form. The 3,350 predecessor comparisons and five malformed inputs are different units and are not added to the receipt count.

8.3 Why the forward oracle is complete on its test fragment

The finite BFS comparator is used only when every transition preserves the total number of tokens. With a fixed initial token total s and dimension d , only finitely many nonnegative vectors sum to s . The oracle explores the entire reachable graph and does not use an arbitrary depth or marking cap.

The exhaustive group enumerates fourteen two-place transitions with input and output totals at most two and exactly equal to one another, ten initial markings with total at most three, and fifteen bad thresholds with total at most four. Their product is $14 \cdot 10 \cdot 15 = 2100$ queries. The random group uses three or four places, small conserved totals, and multiple transitions.

These finite tests check backward search against a computationally different forward algorithm. They do not prove the unbounded algorithm correct. The separately included infinite-state mutex and producer examples exercise that intended setting, and the soundness and termination arguments are given in Section 3. The corpus is synthetic and small, not an industrial Petri benchmark suite.

8.4 What the probability cross-checks establish

For the scalar quadratic family,

$$P(x) = a + (1 - a - b)x + bx^2, \quad a, b > 0, \quad a + b \leq 1,$$

the factorization

$$P(x) - x = (x - 1)(bx - a)$$

gives the exact least fixed point $\min(1, a/b)$. This is an exact independent value check for those 45 systems, not an approximate tolerance comparison.

The coupled contractive systems use a separate high-precision decimal fixed-point iteration to compare numerical values with the exact rational enclosure. Those 80-digit computations are diagnostics, not soundness authorities and not a proof that a returned interval is valid. The exact finite certificate checks authorize the Python result. Likewise, the supercritical cyclic cubic systems exploit symmetry for an algebraic numerical diagnostic, rather than pretending a generic numerical solver selects a least root by itself.

The critical and algebraic examples deliberately exercise places where a naive checker fails: the wrong fixed-point branch, singular deterministic cycles, critical equality without strict contraction, and incorrect factorization. They are not a proof that the checker has no other bugs.

8.5 Mutation and negative controls

The main rejection corpus includes wrong subjects, out-of-range cover indices, altered Newton inverses, a forged jump to the larger fixed point, a false precision claim, incorrect algebraic factors, the deterministic one-child exception, and a budget-exhausted object submitted as though it were a certificate. Each expected rejection is replayable from the supplied JSONL record.

These are designed controls, not a claim about rejection of every possible mutation. A change to an unused statistic, a redundant valid basis row, or an equivalent rational representation need not invalidate the mathematics. Conversely, tests alone do not establish that every malformed input is rejected safely or in bounded resources.

The inverse-free experiment has its own 201 rejection controls: invalid weights and directions on each of the 100 conversions, plus the nonstrict-weight forgery on $P(x) = x$. The main and conversion counts must not be pooled into a single solved-problem rate.

8.6 Search-disabled replay

The standalone replay script imports the checker while installing an import hook that rejects the search module and optional numerical packages. Running with `python-S` also disables site-package initialization. The main replay accepted all 2,360 expected certificates and rejected all 275 expected negative records. Separate replay passes cover the four ablation receipts and the 100 converted receipts with their 201 controls.

This is evidence that acceptance does not depend on rerunning the producer. The checker still shares input representation, validation, and rational serialization code with the producer, and both execute on Python’s runtime and rational arithmetic. It is not a formally verified independent checker and is not a Lean kernel proof.

8.7 Inverse-free format measurements

Dimension	Original bytes	Weighted bytes	Direction of change
1	41,410	55,387	larger
2	35,801	35,287	slightly smaller
3	57,847	40,131	smaller
4	162,855	97,871	smaller
All converted receipts	297,913	228,676	23.2% smaller

These figures count compact UTF-8 JSON serialization of the whole receipt, including its subject and other fields. They do not count Python object memory or a hypothetical Lean expression encoding. Larger rational numerators can outweigh a reduction in the number of rational entries, which is why the report retains per-dimension results rather than advertising a uniform compression factor.

The converter preserves the endpoints and requested-width outcome. Its 471 weighted steps describe the same justified lower-bound progress as the original inverse steps. No search-success improvement is inferred from format conversion alone.

8.8 What was not measured

There was no comparison with `grind`, `Aesop`, `nlinarith`, `LeanHammer`, the existing Forge prototypes, or a production Petri model checker. There was no Lean compilation, no Mathlib cache installation, and no source-level stochastic or resource theorem proved by the delivered package. There was no adversarial parser fuzzing, large-dimensional rational linear algebra study, or industrial benchmark evaluation.

The subsecond timings in the main summary reflect tiny synthetic inputs in this execution environment. They are retained for reproducibility but are not presented as deployment estimates or evidence of a speedup. Certificate size, exactness, and oracle agreement on the stated fragments are the useful measured results here.

9 Implementation order and falsifiable next milestones

9.1 First: one end-to-end unbounded safety theorem

Implement the Petri model, finite safety checker, and its generic induction theorem. Use the supplied mutex certificate as a readable first fixture, but do not stop at proving an abstract net

safe. Define a small resource program and prove the source simulation. The acceptance criterion is a theorem about that original program, with no external solver required during recompilation and with its actual axiom dependencies printed and reviewed.

The intentionally false sibling should permit two owners to acquire the same lock. Its extracted net should produce a concrete violating word, and the source replay should exhibit the violation. A false sibling is important here: a bridge that accidentally drops the acquire operation or misorders the resource coordinates can make every program appear safe while the abstract checker remains correct.

After this works, add nonconservative nets that defeat a finite forward enumeration, larger randomly generated nets, and independent SMPT comparisons. Keep bounded-oracle exhaustion separate from agreement. Search optimizations come after source semantics and certificate replay, not before.

9.2 Second: a weighted-step enclosure theorem

Prove the nonnegative directional remainder lemma directly for sparse monomials. Prove the finite maximum principle and ledger induction. Start with (6); the resulting Lean theorem should say that the *least* fixed point lies in the checked interval. A proof of $P(r) = r$ alone is not the acceptance criterion.

The first negative fixtures should be the wrong fixed-point branch, an upward-rounded lower step that crosses q , an invalid weight, and equality $Jw = w$ falsely used as a strict lower-step witness. Replaying the supplied JSON certificates through a Lean decoder then becomes a useful integration test, rather than the only basis for believing the mathematics.

Only after this abstraction-level theorem works should a branching DSL bridge prove that the interval contains a source termination probability. This sequencing prevents a large probability-library integration from obscuring a small algebraic checker defect.

9.3 Third: exact critical and algebraic conclusions

The critical certificate needs finite graph connectivity and the product-deficit inequality in addition to the weighted comparison machinery. Its first pair of acceptance tests is $P(x) = (1 + x^2)/2$ versus $P(x) = x$. The former must yield probability one; the latter must not. This pair tests the exact condition that distinguishes branching from endless one-child continuation.

The algebraic decoder should then identify (7) using factorization and an isolating interval. Its first extension can reuse the baseline's existing scalar sign-certificate plans for factors with repeated roots or difficult derivative intervals. A new general algebraic-number engine is not required to deliver the first exact irrational result.

9.4 Fourth: synthesis and larger semantic fragments

Once source bridges exist, wire them to retained Leant/Djex candidate graphs for a small provider inventory. Measure source-contract acceptance, counterexample replay, and undecided cases separately. Preserve the original synthesis query, provider identities, and limits; replacing a hard original query by a simpler hand-written program is not acceptance of that query.

For probability systems, implement strongly connected component processing with exact-value substitution first, and interval-parameter transport second. For multiset systems, add dominance indexing and certificate minimization. For Newton search, use sparse exact solves and numerical proposals followed by rational residual checks. A numerical producer can propose d, w directly for (10); it does not have to reconstruct an entire inverse.

Each step has an explicit mathematical contract and an available finite checker target. This makes a failure actionable: unsupported source semantics, no certified lower progress, no prefixed upper candidate, insufficient precision, or resource exhaustion are distinct engineering outcomes.

10 Assessment

The strongest extension is not another collection of local tactics. It is the ability to turn two genuinely infinite questions into small finite obligations of the right logical polarity.

For multiset-state safety, a finite backward basis proves a forward invariant without enumerating all reachable markings. For recursive probabilities, lower histories and prefixed upper vectors distinguish a least solution from arbitrary roots. A weighted maximum principle removes full matrix inverses from many Newton certificates. At criticality, an exact nondegeneracy theorem proves probability one where strict contraction is unavailable. A factor identity and rational interval then make an irrational probability an exact mathematical result rather than a decimal guess.

The delivered package demonstrates those mathematical building blocks with exact arithmetic, independent finite-oracle comparisons where available, explicit negative controls, search-disabled replay, and deterministic reproduction. It does not yet close the source-to-Lean proof chain. The module plan and acceptance criteria identify that remaining work precisely, without claiming that another Python experiment could substitute for it.

A Executable certificate protocol

A.1 Problem objects

Integers are JSON integers, not Booleans. A rational is a pair `[numerator,denominator]` with a strictly positive denominator. The parser canonicalizes rational values and polynomial terms. Vector lengths must agree with the problem dimension.

A Petri input has exactly the following top-level fields:

```
{
  "kind": "petri",
  "initial": [1, 0, 0],
  "transitions": [
    {"pre": [1, 0, 0], "post": [0, 1, 0]},
    {"pre": [0, 1, 0], "post": [1, 0, 0]},
    {"pre": [0, 0, 0], "post": [0, 0, 1]}
  ],
  "targets": [[0, 2, 0]]
}
```

The cubic branching input is:

```
{
  "kind": "pps",
  "polynomials": [
    {"coefficient": [1, 4], "powers": [0]},
    {"coefficient": [3, 4], "powers": [3]}
  ]
}
```

The **subject** field in a receipt is the complete canonical JSON encoding of the normalized problem, not a hash or a pretty-printed summary. The checker receives the problem independently and compares the exact encoding. This binds Python acceptance to the finite mathematical input; a future source bridge must additionally bind that input to the original Lean expression and context.

A.2 Accepted receipt kinds

Kind	Mathematical payload
safe	basis , target_cover , and predecessor_cover ; indices certify the inclusions in Theorem 3.3.
coverable	word , a finite list of transition indices; replayed from the original initial marking.
enclosure	ledger , final lower and upper , requested_bits , and a rechecked target_met . An optional contraction contains weight and rho .
extinction_one	A positive rational weight ; normalization, graph connectivity, mean inequality, and nondegeneracy are recomputed.
algebraic_lfp	A nested accepted scalar enclosure , a nonconstant factor , and a nonzero cofactor , each polynomial listed in ascending degree.

Ledger steps are one of three forms. A **kleene** step supplies **next**. A **newton** step supplies **inverse** and **next**. A **weighted_newton** step supplies **weight**, **direction**, and **next**. All final endpoints are decoded and compared with the endpoints actually obtained from the ledger.

The **unknown** kind is a search outcome and is never accepted as a certificate. Statistics such as expansion counts are not mathematical proof payload. The checker does not trust them, and changing them is not expected to invalidate a correct theorem.

A.3 Input rejection and resource control

The parser and checker validate dimensions, signs, indices, and the exact inequalities stated in the report. They are research code, not a hardened public service. Extremely large integers, exponents, matrices, nested JSON, or ledgers can consume excessive memory or time. Duplicate-key rejection, byte limits, aggregate term-degree limits, and process-level resource isolation belong in a production ingestion layer.

The producer’s work budgets do not replace parser/checker budgets. An externally supplied receipt might be much larger than anything the local producer would emit. In Lean, totality of a recursive checker does not imply that reducing it is affordable. Admission limits therefore need to be specified independently of mathematical validity.

B Checker-oriented proof dependency graph

The Petri lane depends on just three mathematical facts: componentwise predecessor equivalence, complement invariance under a backward-closed set, and induction on finite executions. Dickson’s lemma belongs only to the later search-completeness proof.

The enclosure lane depends on valid polynomial monotonicity, existence of the least fixed point, the prefixed upper lemma, and a ledger induction. The weighted variant additionally uses

the directional remainder and finite maximum principle. It does not require formalizing exact Gaussian elimination or proving the supplied matrix is an inverse.

The critical lane uses normalization, the finite product-deficit inequality, graph propagation of a maximizing ratio, and a strictly positive deficit supplied either by a strict mean row or by actual branching. The algebraic decoder adds a scalar factor identity, interval evaluation soundness, and strict monotonicity from the derivative sign.

This graph is an implementation aid: a failed probability-source bridge should not prevent proving the finite weighted-step theorem, and a missing WQO library lemma should not block an individual Petri safety theorem.

C Files, commands, and status ledger

C.1 Package contents

article/	LaTeX sources and compiled report
prototype/	standard-library-only Python implementations
results/	main runs, replay records, ablations, conversion data
lean/	implementation obligations; no claimed compiled port
tools/	build and full-validation helpers
README.md	entry points and evidence limits
SOURCE-AUDIT.md	inspected sources and pinned revisions

The package includes its own code and generated data, not copies of Forge’s prior proposals or downloaded research papers.

C.2 Replaying without search

From prototype/:

```
python -S replay.py ../results/run-01
python -S replay.py ../results/ablations-01
python -S replay.py ../results/compact-01
```

The output reports the accepted and rejected record counts separately. Do not run the experimental scripts with Python optimization enabled: their internal tests use assertions. The shipped validation helper invokes the ordinary interpreter without `-O`.

C.3 Fresh production and comparison

Each output directory must be new. For example:

```
python -S run_experiments.py --out ../reproduced/main
python -S replay.py ../reproduced/main
python -S ablations.py --out ../reproduced/ablations
python -S compact_experiment.py ../reproduced/main \
  --out ../reproduced/compact
python -S replay.py ../reproduced/compact
```

Main-run failure handling writes a failed summary and traceback to the new run directory before propagating the exception. Recorded results are not overwritten. The original two main runs have byte-identical accepted and rejected corpora; timing fields in their summaries naturally

differ.

C.4 Building the article

From `article/`, run:

```
pdflatex -interaction=nonstopmode -halt-on-error forge-unbounded.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-unbounded.tex
pdflatex -interaction=nonstopmode -halt-on-error forge-unbounded.tex
```

The source uses ordinary pdfLaTeX packages and an inline bibliography. It does not require external fonts, shell escape, or a bibliography processor.

C.5 Evidence status

Item	Status
Backward antichain producer and finite checker	Executed; exact arithmetic.
Rounded Newton enclosure producer and checker	Executed; exact arithmetic.
Inverse-free weighted-step format	Executed; 100 conversions and 201 controls.
Critical extinction and scalar root decoder	Executed in their stated fragments.
Search-disabled corpus replay	Executed.
Same-environment deterministic reproduction	Executed; corpora byte-identical.
Mathematical soundness arguments	Presented in this article; not formalized here.
Lean checker port and theorem compilation	Not run.
Source-to-model compiler correctness	Designed only.
Leant/Djex synthesis integration	Designed only.
Comparison with existing Lean tactics	Not run.
Large-scale or adversarial evaluation	Not run.

References

- [1] Vladimir Reshetnikov et al. *Forge: README and merged design*. Repository revision 58ea206bd7ad501b930add568151217bcc7f82a2. Inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/README.md>.
- [2] Forge, same revision. *Quantitative certificates: probabilities, values, and adversaries*. Merged article, Section 10. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/10-quantitative.tex>.
- [3] Forge, same revision. *Finite certificates for unbounded behaviour*. Beginning of merged article Section 6, inspected source lines 1–240. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/article/sections/06-closure.tex>.
- [4] Forge, same revision. *Ideas from Leant and Djex*. <https://github.com/VladimirReshetnikov/Forge/blob/58ea206bd7ad501b930add568151217bcc7f82a2/docs/IDEAS-FROM-LEANT-DJEX.md>.
- [5] Vladimir Reshetnikov et al. *Leant*. README and current commit metadata; revision 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/tree/6bf05ad78c467989e68290f2d08bbbed40802d485>.

- [6] Vladimir Reshetnikov et al. *Djex*. README and current commit metadata; revision `e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5`. <https://github.com/VladimirReshetnikov/Djex/tree/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5>.
- [7] Alain Finkel and Philippe Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science* 256(1–2), 63–92, 2001. DOI: 10.1016/S0304-3975(00)00102-X.
- [8] Pierre-Alain Reynier and Frédéric Servais. Minimal coverability set for Petri nets: Karp and Miller algorithm with pruning. *Fundamenta Informaticae* 122(1–2), 1–30, 2013. DOI: 10.3233/FI-2013-781.
- [9] Kousha Etessami and Mihalis Yannakakis. Recursive Markov chains, stochastic grammars, and monotone systems of nonlinear equations. *Journal of the ACM* 56(1), 2009. DOI: 10.1145/1462153.1462154. Author institution record: <https://www.research.ed.ac.uk/en/publications/recursive-markov-chains-stochastic-grammars-and-monotone-systems-/>.
- [10] Javier Esparza, Stefan Kiefer, and Michael Luttenberger. Computing the least fixed point of positive polynomial systems. *SIAM Journal on Computing* 39(6), 2282–2335, 2010. DOI: 10.1137/090749591. <https://arxiv.org/abs/1001.0340>.
- [11] Kousha Etessami, Alistair Stewart, and Mihalis Yannakakis. A polynomial time algorithm for computing extinction probabilities of multitype branching processes. *SIAM Journal on Computing*, 2017. DOI: 10.1137/16M105678X.
- [12] Alistair Stewart, Kousha Etessami, and Mihalis Yannakakis. Upper bounds for Newton’s method on monotone polynomial systems, and P-time model checking of probabilistic one-counter automata. 2013 preprint. <https://arxiv.org/abs/1302.3741>.
- [13] Mathlib contributors. *Mathlib.Order.FixedPoints*. Current generated documentation, inspected 15 September 2026; API discovery, not a compiled dependency pin. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Order/FixedPoints.html.
- [14] Mathlib contributors. *Mathlib.Order.WellFoundedSet*. Current generated documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Order/WellFoundedSet.html.
- [15] FLINT contributors. *fmpq_mat.h: matrices over the rational numbers*. Official documentation, indexed contents inspected 15 September 2026. https://flintlib.org/doc/fmpq_mat.html.
- [16] FLINT contributors. *fmpq_mpoly.h: multivariate polynomials over the rational numbers*. Official documentation. https://flintlib.org/doc/fmpq_mpoly.html.
- [17] FLINT contributors. *fmpz_poly_factor.h: factorisation of polynomials over the integers*. Official documentation. https://flintlib.org/doc/fmpz_poly_factor.html.
- [18] FLINT contributors. *qqbar.h: algebraic numbers represented by minimal polynomials*. Official documentation. <https://flintlib.org/doc/qqbar.html>.
- [19] Nicolas Amat et al. *SMPT: Satisfiability Modulo Petri Nets*. Official source repository and README, inspected 15 September 2026. <https://github.com/nicolasAmat/SMPT>.